

Cross-Environmental Website Fingerprinting

Jianfeng Li^{†§}, Dongliang Wang^{†§}, Yixuan Liu^{†§}, Yifei Gao^{†§},
Xiaorong Zhang^{†§}, Zheng Lin^{†§}, Xiaobo Ma^{†§*}, Xiapu Luo[‡] and Xiaohong Guan^{†§}
[†]MOE Key Lab for Intelligent Networks and Network Security, Xi'an Jiaotong University, Xi'an, China
[§]Faculty of Electronic and Information Engineering, Xi'an Jiaotong University, Xi'an, China
[‡]Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China

Abstract—Despite the widespread adoption of encryption, such as TLS, encrypted proxies, and Tor, website fingerprinting (WF) has long been proven to be able to recognize websites from encrypted traffic. However, existing WF methods were generally developed and evaluated under the implicit assumption that traffic samples for training and recognition are captured in the same environment. When applied to diverse environments affected by practical factors, such as various browsers and proxy software, they will be hampered by three-fold challenges: i) feature drift, ii) sampling dilemma, and iii) few-shot generalization. None of existing WF methods can fully address them. In this paper, we take the *first* step to cross-environmental WF and advance a systematic framework, dubbed X-EPRINT, to tackle the above challenges. X-EPRINT generates cross-environmentally invariant features to address feature drift. It mitigates sampling dilemma via potential-aware traffic resampling. X-EPRINT capitalizes on inter-flow data augmentation to solve few-shot generalization. We conduct extensive experiments to evaluate X-EPRINT. The experimental results demonstrate that X-EPRINT achieves a robust performance in zero-shot cross-environmental recognition, with an F1-score of 0.719, which is 58.4% higher than the top-performing baseline method. It also attains an F1-score of 0.925 in 3-shot recognition, fulfilling few-shot environment adaptation.

I. INTRODUCTION

Encrypted communication protocols, such as transport layer security (TLS), have become the de facto standard for web-based services because they ensure the confidentiality and integrity of transmitted data [1], [2]. The emergence of anonymity networks (e.g., Tor [3]) and encrypted proxies [4], [5], further enhances online user privacy by concealing the IP address of remote server through traffic relaying [6]. This concealment hinders direct identification of the websites a user visits from remote IP addresses. Nevertheless, website fingerprinting (WF), a crucial family of passive traffic analysis methods, is immune to the above encryption techniques, since it neither relies on remote IP addresses nor requires decrypting traffic. Instead, it recognizes monitored website traffic by training (deep) machine learning models that extract features from packet direction, size, and timing [7]–[20].

The effectiveness of WF methods fundamentally depends on their ability to learn the traffic patterns of monitored websites. Unfortunately, traffic patterns are likely to vary across diverse environments influenced by practical factors, such as software and network conditions. This variability is particularly noticeable with encrypted proxies, where users access websites using a wide spectrum of browsers and proxy

software. Changes in the browser and proxy software can lead to variations in the traffic patterns of the same website.

Existing WF methods were generally developed and evaluated under the implicit assumption that traffic samples for model training and testing were captured in the same environment [7], [9]–[18]. When applied to diverse environments, they will be hampered by three-fold challenges.

C1: Feature Drift. Different browsers vary in HTTP headers, rendering engines, and plugins, causing feature drift in website traffic patterns. For instance, unique User-Agent strings and HTTP header sizes differ across browsers [21], [22]. Feature drift can also result from diverse proxy software, which vary in encryption protocols, packet formats, and security strategies. For example, different proxy software might use varying packet padding schemes for traffic obfuscation [23]. As typical concept drift, feature drift is still an open problem for machine learning (ML) [24]–[26]. WF methods heavily rely on (deep) ML models [7]–[20], thereby susceptible to feature drift.

C2: Sampling Dilemma. To adapt existing WF methods to diverse environments, one straightforward solution is to collect traffic samples from monitored websites for each environment and train environment-specific models. Unfortunately, this solution is prohibitively costly for encrypted proxies, due to the hundreds of individual browsers and proxy software options available [27]. A more cost-effective solution involves training the model with samples from the same environment as the recognition stage, if necessary. However, it is unclear a priori which websites are truly necessary. Selectively sampling for a few websites might miss those present in recognition stage, increasing false-negative risk. Conversely, including all monitored websites escalates sample collection overhead.

C3: Few-Shot Generalization. When deployed in a new environment, recognition models must quickly adapt and effectively handle unseen samples from monitored websites with minimal training. However, WF methods are generally data-hungry, demanding sufficient and representative labeled traffic for model training [11], [13], [17]. Collecting numerous training samples will entail considerable resource consumption.

To tackle the above challenges, we advance a systematic framework, dubbed X-EPRINT, to take the *first* step to cross-environmental website fingerprinting on encrypted proxies.

X-EPRINT first carries out zero-shot cross-environmental recognition, training the model on samples from a source environment to recognize monitored websites in the target environment. To tackle **C1**, X-EPRINT automatically generates

*The corresponding author.

cross-environmentally *invariant features* to minimize deviations between the traffic patterns observed in recognition stage and those learned during training. We formulate invariant feature generation as a bilevel combinatorial optimization and efficiently solve it via adaptive Thompson sampling.

To further improve recognition accuracy, X-EPRINT conducts *potential-aware traffic resampling* in the target environment. Specifically, we estimate error rates in recognizing monitored websites by leveraging confidence scores obtained in zero-shot cross-environmental recognition. To tackle **C2**, we optimize resampling effectiveness by prioritizing monitored websites with higher error rates, while adhering to resource constraints.

X-EPRINT addresses **C3** via few-shot environment adaptation. For monitored websites, we resample only a few traffic samples as needed in the target environment. These samples are utilized to train environment-specific models in favor of a higher recognition accuracy. To this end, we capitalize on both intra-flow and inter-flow features from limited samples to develop a bilevel recognition model, incorporating *inter-flow data augmentation* to mitigate overfit.

X-EPRINT markedly alleviates the burden of traffic sample collection, enabling fast deployment across diverse environments. We summarize our contributions as follows.

- To the best of our knowledge, our work presents the *first* cross-environmental website fingerprinting method on encrypted proxies, capable of adapting to diverse environments affected by various browsers and proxy software.
- We address three challenges in designing X-EPRINT. To address feature drift, we generate cross-environmentally invariant features automatically. To resolve sampling dilemma, we introduce potential-aware traffic resampling for enhanced efficiency. For few-shot generalization, we utilize inter-flow data augmentation to mitigate overfitting risks.
- We evaluate X-EPRINT through extensive experiments involving eighteen scenarios. X-EPRINT shows substantial advantage over baseline methods. It achieves an average F1-score of 0.719 in zero-shot cross-environmental recognition, which is 58.4% higher than that of the top-performing baseline method. Its F1-score is further improved to 0.925 via resampling just three training samples in the target environment.

II. OVERVIEW

We start with a quantitative analysis of feature drift in cross-environmental recognition, define the problem addressed in this paper, and then outline the workflow of X-EPRINT.

A. Understanding Cross-Environmental Feature Drift

We compare the similarity of network flows from a website within the same environment to those across different environments by calculating the dynamic time warping (DTW) distance, a commonly used metric to quantify sequence similarity [28], between two network flows. To improve comparison efficiency, we group network flows by remote hostnames and calculate the DTW distance between flows in logarithmic burst representation for the same hostname. The minimum pairwise DTW distance for network flows is calculated in both

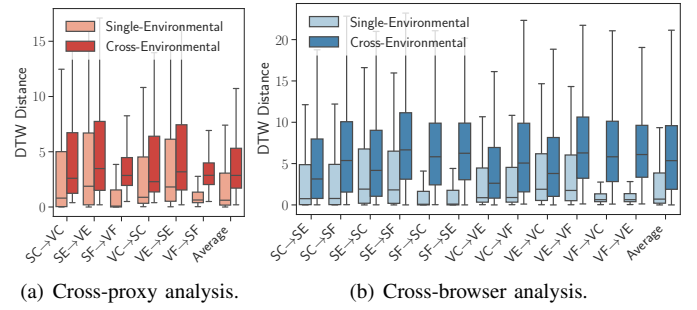


Fig. 1: Comparison of DTW distances: cross-environmental vs. single-environmental network flows for the same hostname.

single-environmental and cross-environmental settings. We use minimum distances instead of average distances because they more accurately reflect the proximity between a test sample and its nearest training sample in the feature space. Fig. 1 presents the results based on 200 randomly chosen websites from CE-450×6 dataset (§ VI-A). In Fig. 1, different categories correspond to various cross-environmental scenarios. For instance, the category labeled “VF → VC” denotes VF as the source and VC as the target environment (detailed in Table I). Single-environmental DTW distances are measured between flows within the same source environment, while cross-environmental DTW distances compare flows from the target environment to those from the source. For all scenarios, cross-environmental DTW distances are greater than single-environmental DTW distances, indicating substantial cross-environmental feature drift that can severely degrade the performance of WF methods.

B. Problem Definition

In this paper, we define an environment as the specific combination of browser and proxy software that a user employs to access websites. Assume that $\mathbb{W}$ is a monitored website. We have collected its training sample set $\mathcal{D}_{\mathbb{W}}^s$ in Θ_s (source environment). However, the encrypted traffic $\mathcal{T}$ captured in the recognition stage is from Θ_t (target environment) where we didn’t collect its training samples. We aim to cross-environmentally recognize $\mathbb{W}$ from $\mathcal{T}$ by leveraging the model trained on $\mathcal{D}_{\mathbb{W}}^s$. Given the results of cross-environmental recognition, we assess the necessity of resampling traffic samples of $\mathbb{W}$ from Θ_t . If necessary, we will collect a very small number of traffic samples of $\mathbb{W}$ in Θ_t for model training and subsequently re-recognize $\mathbb{W}$ in favor of higher accuracy. We denote by $\mathcal{W}_s$ (resp. $\mathcal{W}_t$) a set comprised of websites for which we have collected traffic samples in Θ_s (resp. Θ_t). We assume that $\mathcal{W}_s \cap \mathcal{W}_t \neq \emptyset$. Traffic samples of websites in $\mathcal{W}_s \cap \mathcal{W}_t$ are informative in characterizing the variation of traffic features from Θ_s to Θ_t , facilitating the recognition of $\mathbb{W}$.

C. Workflow of X-EPRINT

X-EPRINT aims to recognize monitored websites in cross-environmental scenarios by tackling feature drift (**C1**), sampling dilemma (**C2**), and few-shot generalization (**C3**). Fig. 2 illustrates its three-stage workflow for achieving this goal.

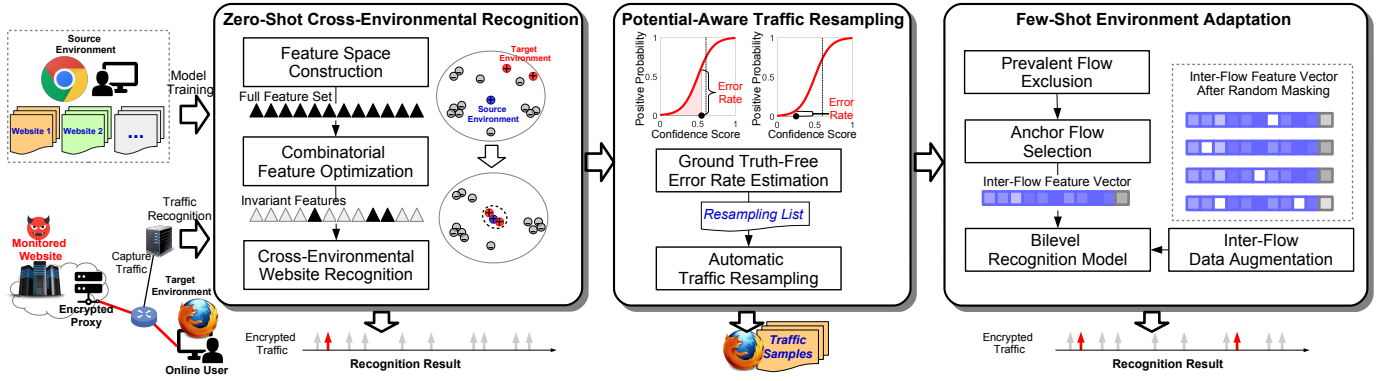


Fig. 2: The workflow of X-EPRINT.

1) **Zero-Shot Cross-Environmental Recognition:** In training stage, given any source environment Θ_s and target environment Θ_t , X-EPRINT constructs the feature space spanned by the feature set Γ and generates cross-environmentally invariant features $\Gamma_{s,t}^* \subset \Gamma$ via combinatorial feature optimization. For each monitored website, say $\mathbb{W}$, if $\mathbb{W} \in \mathcal{W}_s$ but $\mathbb{W} \notin \mathcal{W}_t$, we train a cross-environmental recognition model $\Phi_{\mathbb{W}}^{s,t}$ with traffic samples from $\mathcal{D}_{\mathbb{W}}^s$. Since $\Phi_{\mathbb{W}}^{s,t}$ extracts the feature vector of network flows by focusing solely on the features within $\Gamma_{s,t}^*$, X-EPRINT effectively addresses **C1**. In recognition stage, X-EPRINT first identifies Θ_t in which $\mathcal{T}$ is generated and then employs $\Phi_{\mathbb{W}}^{s,t}$ to recognize $\mathbb{W}$ from $\mathcal{T}$. Note that cross-environmental recognition is unnecessary if $\mathbb{W} \in \mathcal{W}_t$ because training samples of $\mathbb{W}$ are already available in Θ_t .

2) **Potential-Aware Traffic Resampling:** X-EPRINT further improves recognition accuracy via active learning. Unlike classic active learning, where experts relabel samples with uncertain predictions [25], [29], our problem involves encrypted traffic that cannot be decrypted to obtain ground truth. Therefore, X-EPRINT enhances recognition accuracy by resampling training samples of monitored websites in Θ_t , if necessary. To this end, we estimate the error rate in recognizing $\mathbb{W}$ based on the confidence score delivered by $\Phi_{\mathbb{W}}^{s,t}$, without the need of ground truth. X-EPRINT prioritizes the resampling of monitored websites with higher estimated error rates to address **C2**.

3) **Few-Shot Environment Adaptation:** To cover more monitored websites, X-EPRINT only resamples a small number of training samples for each monitored website. We extract both intra-flow and inter-flow features from new samples and construct a bilevel recognition model to re-recognize $\mathbb{W}$ from $\mathcal{T}$. At the low level, we extract intra-flow features and train classifiers to i) exclude network flows prevalent across various websites and ii) identify the pattern of anchor flows (i.e., representative network flows of $\mathbb{W}$). At a high level, we synthesize results from low-level classifiers to derive inter-flow feature vectors for recognizing $\mathbb{W}$. X-EPRINT enhances model robustness and mitigates overfitting of limited samples via inter-flow data augmentation, effectively addressing **C3**.

III. ZERO-SHOT CROSS-ENVIRONMENTAL RECOGNITION

We elaborate on how X-EPRINT cross-environmentally recognizes $\mathbb{W}$. Given that no traffic samples of $\mathbb{W}$ from the target

environment Θ_t are used in model training, we refer to it as zero-shot cross-environmental recognition.

A. Encrypted Flow Preprocessing

Encrypted proxies typically operate at the transport layer. In both training and recognition stages, the captured encrypted traffic can be divided into a series of network flows identified by the 5-tuple, including source/destination IP address/port and transport protocol.

For each flow, we preprocess it in three steps. First, we eliminate TCP retransmission packets to improve feature robustness against packet loss. Second, we remove control packets to mitigate the noises introduced by TCP congestion control. Last, we represent a network flow using burst representation by merging consecutive packets arriving in the same direction into a burst. Formally, we denote by $\mathbf{x}$ a network flow, where $\mathbf{x}(k)$ represents the number of bytes in the k th burst of $\mathbf{x}$.

B. Feature Space Construction

To simplify feature extraction, we pad or truncate $\mathbf{x}$ to a fixed length of n . We set $n = 16$ as it exceeds the burst count for most network flows. We construct a feature space of network flows using various operators:

- **Unary Operator.** Their inputs consist of a single burst. We involve unary operators: i) the identity function i.e., $\text{id}(\mathbf{x}(k)) = \mathbf{x}(k)$ and ii) the logarithm of the absolute value, i.e., $\text{alog}(\mathbf{x}(k)) = \log(|\mathbf{x}(k)|)$.
- **Binary Operator.** Their inputs consist of two bursts. We consider the difference function, i.e., $\text{diff}(\mathbf{x}(i), \mathbf{x}(j)) = |\mathbf{x}(i)| - |\mathbf{x}(j)|$ for $1 \leq i < j \leq n$.
- **Polyadic Operator.** Their inputs consist of all bursts in $\mathbf{x}$. We involve polyadic operators: i) the log-scale histogram function $\text{hist}(\mathbf{x})$, ii) the burst number in $\mathbf{x}$, iii) the 5th, 25th, 50th, 75th, 95th percentile for burst sizes, and iv) statistic functions $\text{max}(\mathbf{x})$, $\text{min}(\mathbf{x})$, $\text{mean}(\mathbf{x})$, and $\text{std}(\mathbf{x})$.

We denote Γ as the set of all features extracted by the operators, spanning a 167-dimensional feature space.

C. Invariant Feature Generation

In this paper, a feature is deemed cross-environmentally invariant if it remains stable across environments and contributes to distinguishing websites. Given a source environment Θ_s

and a target environment Θ_t , multiple cross-environmentally invariant features may exist that are both interrelated and potentially redundant. Let $\Gamma_{s,t}^* \subset \Gamma$ be the invariant feature subset. We derive $\Gamma_{s,t}^*$ with the aid of websites in $\mathcal{W}_s \cap \mathcal{W}_t$. Specifically, combining network flows in $\cup_{w \in \mathcal{W}_s \cap \mathcal{W}_t} \mathcal{D}_w^s$ yields a flow list $\mathbf{X}^s$ and a label list $\mathbf{Y}^s$, where $\mathbf{Y}^s(i)$ indicates the website that generates the network flow $\mathbf{X}^s(i)$. Similarly, we obtain the flow list $\mathbf{X}^t$ and label list $\mathbf{Y}^t$ for Θ_t . We derive $\Gamma_{s,t}^*$ by solving the bilevel combinatorial optimization below.

$$\begin{aligned} & \max_{\Gamma_{s,t}} E(\Gamma_{s,t}) \\ \text{s.t. } & E(\Gamma_{s,t}) = \text{F1-Score}(\mathbf{Y}^t, [f_{\theta^*}(\mathbf{g}(\mathbf{X}^t(i), \Gamma_{s,t}))]_{i=1}^{|\mathbf{X}^t|}), \\ & \theta^* = \arg \min_{\theta} L(\mathbf{Y}^s, [f_{\theta}(\mathbf{g}(\mathbf{X}^s(i), \Gamma_{s,t}))]_{i=1}^{|\mathbf{X}^s|}), \\ & K_{\min} \leq |\Gamma_{s,t}| \leq K_{\max}, \end{aligned} \quad (1)$$

where $\Gamma_{s,t}$ is the feature subset, $E(\Gamma_{s,t})$ is defined as the macro F1-score, $\mathbf{g}(\mathbf{x}, \Gamma_{s,t})$ generates the feature vector of a network flow $\mathbf{x}$ by extracting features in $\Gamma_{s,t}$, f_{θ} is a multiclass random forest (RF) classifier trained on $\{\mathbf{X}^s, \mathbf{Y}^s\}$ by implicitly minimizing the loss function L , i.e., Gini impurity, and $K_{\min}$ (resp. $K_{\max}$) is the lower bound (resp. upper bound) of $|\Gamma_{s,t}|$. To efficiently solve the optimization in Eqn. (1), we recast it as a combinatorial multi-armed bandit problem, where $\Gamma_{s,t}$ can be viewed as the combinatorial arms. This is then solved using Thompson sampling (TS) [30]–[32]. However, existing TS-based methods cannot be directly applied to our problem, because they require the number of combinatorial arms to be predetermined whereas $|\Gamma_{s,t}^*|$ is unknown a priori. To solve this problem, we propose adaptive Thompson sampling.

• **Dummy Feature Injection.** We add $K_{\max} - K_{\min}$ dummy features to Γ , creating an expanded feature set Γ' . Dummy features are specified to be constants across network flows for all websites. Therefore, they neither positively nor negatively affect the classifier f_{θ} and can be excluded if selected into $\Gamma_{s,t}^*$. We set the number of combinatorial arms to be $K_{\max}$ and exclude dummy features in $\Gamma_{s,t}^*$. By doing so, the feature number in $\Gamma_{s,t}$ is adaptively adjusted.

Let γ_i be the i th feature in Γ' . We denote by $p_i = \mathbb{P}(\Gamma_{s,t} = \Gamma_{s,t}^* \mid \gamma_i \in \Gamma_{s,t})$ the conditional probability that $\Gamma_{s,t}$ is $\Gamma_{s,t}^*$ if γ_i is selected to $\Gamma_{s,t}$. If $\gamma_i \in \Gamma_{s,t}$, whether $\Gamma_{s,t}$ is $\Gamma_{s,t}^*$ can be viewed as Bernoulli trial with a success probability of p_i . We utilize the beta distribution $\text{Beta}(\alpha_i, \beta_i)$ as the prior of p_i since it is the conjugate prior for the Bernoulli distribution. Below we iteratively optimize $E(\Gamma_{s,t})$ from **S1** to **S4**.

S1: Sample Bernoulli parameter. For all features in Γ' , we sample their Bernoulli parameters from a beta distribution. We denote by $\hat{p}_i \sim \text{Beta}(\alpha_i, \beta_i)$ the Bernoulli parameter of γ_i .

S2: Construct feature subset. To construct $\Gamma_{s,t} = \{\gamma_j'\}_{j=1}^{K_{\max}}$, we conduct random feature selection (resp. optimal feature selection) with a probability of ϵ_r (resp. $1 - \epsilon_r$). For the random feature selection, we randomly choose $K_{\max}$ features from Γ' to construct $\Gamma_{s,t}$. As for the optimal feature selection, we aim to optimally select features. Note that optimizing $E(\Gamma_{s,t})$ is equivalent to maximizing $P(\Gamma_{s,t}^*) = \mathbb{P}(\Gamma_{s,t} = \Gamma_{s,t}^* \mid \gamma_1' \in \Gamma_{s,t}, \dots, \gamma_{K_{\max}}' \in \Gamma_{s,t})$. Disregarding the dependencies

among different features, we obtain $P(\Gamma_{s,t}^*) \propto \prod_{j=1}^{K_{\max}} \mathbb{P}(\gamma_j' \in \Gamma_{s,t} \mid \Gamma_{s,t} = \Gamma_{s,t}^*)$. Without loss of generality, we assume that $\gamma_j' = \gamma_i$ and thus obtain $\mathbb{P}(\gamma_j' \in \Gamma_{s,t} \mid \Gamma_{s,t} = \Gamma_{s,t}^*) = \frac{\mathbb{P}(\Gamma_{s,t} = \Gamma_{s,t}^*)}{\mathbb{P}(\gamma_i \in \Gamma_{s,t})} p_i = \frac{\mathbb{P}(\Gamma_{s,t} = \Gamma_{s,t}^*)}{K_{\max}} p_i \approx \frac{\mathbb{P}(\Gamma_{s,t} = \Gamma_{s,t}^*)}{K_{\max}} \hat{p}_i$. As such, we maximize $P(\Gamma_{s,t}^*)$ by selecting top $K_{\max}$ features with the largest Bernoulli parameters into $\Gamma_{s,t}$.

S3: Assess feature quality. Given $\Gamma_{s,t}$, we generate the feature vector of $\mathbf{x}$ as $\mathbf{g}(\mathbf{x}, \Gamma_{s,t})$. We train f_{θ} on $\{\mathbf{X}^s, \mathbf{Y}^s\}$ but test it on $\{\mathbf{X}^t, \mathbf{Y}^t\}$ to compute $E(\Gamma_{s,t})$. A high $E(\Gamma_{s,t})$ suggests $\Gamma_{s,t}$ are not only effective in differentiating websites but are also invariant from environment Θ_s to Θ_t . $E(\Gamma_{s,t})$ will be recorded for feature quality comparison.

S4: Update prior distribution. If $E(\Gamma_{s,t})$ equals or exceeds the highest recorded performance, we refer to $\Gamma_{s,t}$ as the newest optimal feature subset. If so, for $\gamma_i \in \Gamma_{s,t}$, we update the prior $\text{Beta}(\alpha_i, \beta_i)$ by incrementing α_i (the success count) by 1. If not, we increment β_i (the failure count) by 1.

We repeat steps **S1** to **S4** until $E(\Gamma_{s,t})$ converges or the maximum number of iterations is reached. Removing dummy features (if present) from the $\Gamma_{s,t}$ with the highest $E(\Gamma_{s,t})$ yields $\Gamma_{s,t}^*$. To prevent $\Gamma_{s,t}^*$ from getting trapped in local optima, we propose an ensemble optimization strategy:

• **Ensemble Optimization.** Executing the iterative optimization process m_f rounds yields m_f optimal feature subsets, denoted by $\Gamma_{s,t}^{*(k)}$ for $1 \leq k \leq m_f$. Each $\Gamma_{s,t}^{*(k)}$ acts as a candidate of the final $\Gamma_{s,t}^*$. To introduce more diversity, we further generate candidates via feature stability analysis. Specifically, given a stability threshold $0 < \tau \leq 1$, we derive a feature subset $\Gamma(\tau) = \{\gamma_i \in \Gamma \mid \frac{1}{m_f} \sum_{k=1}^{m_f} \mathbf{1}_{\gamma_i \in \Gamma_{s,t}^{*(k)}}(\gamma_i) \geq \tau\}$. We derive $\Gamma_{s,t}^*$ as the $\Gamma_{s,t}$ with the highest $E(\Gamma_{s,t})$ from $\{\Gamma_{s,t}^{*(k)}\}_{k=1}^{m_f} \cup \{\Gamma(\tau) \mid 0 < \tau \leq 1, K_{\min} \leq |\Gamma(\tau)| \leq K_{\max}\}$.

D. Cross-Environmental Recognition

After capturing the encrypted traffic $\mathcal{T}$, X-EPRINT first identifies the target environment corresponding to $\mathcal{T}$.

• **Environment Identification.** For each environment, say Θ_a , we train a binary RF classifier $\Upsilon_a(\mathbf{x})$ to determine how likely the flow $\mathbf{x}$ is from Θ_a . To train Υ_a , we construct the positive training set $\cup_{w \in \mathcal{W}_a} \mathcal{D}_w^a$ and the negative training set $\cup_{e \neq a} \cup_{w \in \mathcal{W}_e} \mathcal{D}_w^e$. In recognition stage, we compute $v_a(\mathcal{T}) = \frac{\sum_{\mathbf{x}_j \in \mathcal{T}} \sum_{\mathbf{x}_i \in \mathcal{X}_j} \Upsilon_a(\mathbf{x}_i)}{\sum_{\mathbf{x}_j \in \mathcal{T}} |\mathcal{X}_j|}$ to quantify how likely $\mathcal{T}$ is from Θ_a .

We identify the target environment $\Theta_t = \arg \max_{\Theta_a} v_a(\mathcal{T})$.

• **Confidence Score.** If $\mathbb{W} \in \mathcal{W}_s$ but $\mathbb{W} \notin \mathcal{W}_t$, we construct the cross-environmental recognition model $\Phi_{\mathbb{W}}^{s,t}$ as a binary RF classifier with probabilistic output to predict how likely a network flow is from $\mathbb{W}$. $\Phi_{\mathbb{W}}^{s,t}$ is trained on samples from Θ_s but recognizes $\mathbb{W}$ from Θ_t . If multiple Θ_s are available, we select $\Theta_{s^*} = \arg \max_{\Theta_s} E(\Gamma_{s,t}^*)$ to maximize the expected performance. When a user visits a website, network flows are densely established for data transmission. Conversely, traffic originating from different websites is typically separated by obvious time gaps between the start time of network flows. Therefore, we set a time threshold η to separate traffic samples from different websites in recognition stage. If the time gap

between the start times of two flows exceeds η , they are split into different traffic samples. Similar strategy has been widely used in existing WF methods [9], [10], [12]–[18]. For a sample $\mathcal{X}_j \in \mathcal{T}$, we compute $s_j = \sum_{\mathbf{x} \in \mathcal{X}_j} \frac{\Phi_{\mathbb{W}}^{s,t}(\mathbf{g}(\mathbf{x}, \Gamma_{s,t}^*))}{|\mathcal{X}_j|}$ as the confidence score that $\mathcal{X}_j$ is from $\mathbb{W}$. Given a recognition threshold ϵ_z , we recognize that $\mathcal{X}_j$ is from $\mathbb{W}$ if $s_j \geq \epsilon_z$.

IV. POTENTIAL-AWARE TRAFFIC RESAMPLING

Given resource constraints such as bandwidth, CPU time, and storage, we aim to maximize the overall improvement in recognition accuracy by strategically cherry-picking the monitored websites that most urgently require traffic resampling.

• **Error Rate Estimation.** To prioritize the monitored websites with the highest potential for accuracy enhancement if resampling them in Θ_t , we estimate the error rate for each monitored website from confidence scores obtained in § III-D, without relying on ground truth. We denote by $p_{\mathbb{W}}(s_j) = \mathbb{P}(y_j = \mathbb{W} | s_j)$ the conditional probability that $\mathcal{X}_j$ originates from $\mathbb{W}$ given its confidence score s_j . The expected error rate for $\mathbb{W}$ in the zero-shot recognition can be represented by

$$\mathcal{E}_{\mathbb{W}}^t = \frac{1}{|\mathcal{T}|} \sum_{j=1}^{|\mathcal{T}|} p_{\mathbb{W}}(s_j) \cdot \mathbf{1}_{s_j < \epsilon_z} + [1 - p_{\mathbb{W}}(s_j)] \cdot \mathbf{1}_{s_j \geq \epsilon_z} (s_j). \quad (2)$$

To calculate $\mathcal{E}_{\mathbb{W}}^t$, we derive $p_{\mathbb{W}}(s_j) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 s_j)}}$ with a logistic model, where parameters θ_0 and θ_1 can be learned from websites in $\mathcal{W}_s \cap \mathcal{W}_t$. Specifically, for any $w \in \mathcal{W}_s \cap \mathcal{W}_t$, we construct a positive sample set $\mathcal{P}_w^{s,t} = \{\sum_{\mathbf{x} \in \mathcal{X}_j} \frac{\Phi_{\mathbb{W}}^{s,t}(\mathbf{g}(\mathbf{x}, \Gamma_{s,t}^*))}{|\mathcal{X}_j|} \mid \mathcal{X}_j \in \mathcal{D}_w^t\}$ to include confidence scores of traffic samples from w and a negative sample set $\mathcal{N}_w^{s,t} = \{\sum_{\mathbf{x} \in \mathcal{X}_j} \frac{\Phi_{\mathbb{W}}^{s,t}(\mathbf{g}(\mathbf{x}, \Gamma_{s,t}^*))}{|\mathcal{X}_j|} \mid \mathcal{X}_j \in \cup_{w' \in \mathcal{W}_t \setminus \{w\}} \mathcal{D}_{w'}^t\}$ to include confidence scores of traffic samples from other websites. Merging positive (resp. negative) sample sets of websites in $\mathcal{W}_s \cap \mathcal{W}_t$ yields $\mathcal{P}^{s,t}$ (resp. $\mathcal{N}^{s,t}$). We derive $\hat{\theta}_0, \hat{\theta}_1 = \arg \max_{\theta_0, \theta_1} \frac{1}{|\mathcal{P}^{s,t}|} \sum_{s_i \in \mathcal{P}^{s,t}} \ln \frac{1}{1 + e^{-(\theta_0 + \theta_1 s_i)}} + \frac{1}{|\mathcal{N}^{s,t}|} \sum_{s_i \in \mathcal{N}^{s,t}} \ln \frac{e^{-(\theta_0 + \theta_1 s_i)}}{1 + e^{-(\theta_0 + \theta_1 s_i)}}$.

• **Automatic Resampling.** Let $\mathcal{M}$ be the set containing all monitored websites. We construct a resampling list $\mathcal{L}_d$ to arrange websites in $\mathcal{M} \setminus \mathcal{W}_t$ in descending order by their error rates estimated in Eqn. (2). We establish testbeds involving various environments, e.g., different combinations of browser and proxy software. Given the target environment Θ_t , we resample traffic samples of websites according to their order in $\mathcal{L}_d$. It is worth noting that not all websites in $\mathcal{L}_d$ will be resampled, as this is constrained by available resources. To perform traffic resampling, we automate visits to monitored websites in $\mathcal{L}_d$ using Selenium [33] and capture encrypted traffic with Tcpdump [34]. We also record the remote host linked to each network flow from proxy logs, used only in training, not during recognition, as detailed in § V.

V. FEW-SHOT ENVIRONMENT ADAPTATION

X-EPRINT constructs a bilevel recognition model to fully exploit intra-flow and inter-flow features from limited samples.

Network flows associated with certain server hosts, e.g., thirty-party APIs, are frequently observed across different

websites. We refer to these network flows as *prevalent flows*. Since the presence of prevalent flows could introduce noise and increase the risk of false positives, we first filter out them.

• **Prevalent Flow Exclusion.** Recall that we record remote hosts of network flows when collecting traffic samples. We denote by $\mathcal{H}^t$ a set of remote hosts associated with websites $\mathcal{W}_t$. For a website $w \in \mathcal{W}_t$, if there exist some network flows in $\mathcal{D}_w^t$ linked to the remote host h , we refer to w as the relevant website for h . Let $\mathcal{G}_h^t$ be a set containing all relevant websites for h . A higher $\frac{|\mathcal{G}_h^t|}{|\mathcal{W}_t|}$ indicates h is more prevalent among various websites. Given a prevalence threshold η_p , we construct the prevalent host set $\mathcal{P}^t = \{h \in \mathcal{H}^t \mid \frac{|\mathcal{G}_h^t|}{|\mathcal{W}_t|} \geq \eta_p\}$ for Θ_t . Network flows associated with hosts in $\mathcal{P}^t$ are prevalent flows. We train a binary RF classifier $Q^t(\mathbf{x})$ on $\cup_{w \in \mathcal{W}_t} \mathcal{D}_w^t$ to identify prevalent flows and exclude them from $\mathcal{T}$.

• **Anchor Flow Selection.** Network flows linked to hosts observed significantly more frequently for $\mathbb{W}$ than for other websites can be strong indicators of $\mathbb{W}$'s presence. Let $\mathcal{H}_{\mathbb{W}}$ be the set of all remote hosts associated with $\mathbb{W}$, excluding prevalent hosts. To quantify how important h indicates $\mathbb{W}$'s presence, we calculate $\text{tf-idf}(h, \mathcal{X}_j, \mathcal{G}_h^t) = \text{tf}(h, \mathcal{X}_j) \cdot \text{idf}(h, \mathcal{G}_h^t)$, where $\text{tf}(h, \mathcal{X}_j) = \frac{|\{i \mid \text{host}(\mathbf{x}_i) = h, \mathbf{x}_i \in \mathcal{X}_j\}|}{|\mathcal{X}_j|}$ is the term frequency and $\text{idf}(h, \mathcal{G}_h^t) = \log \frac{|\mathcal{W}_t| + 1}{|\mathcal{G}_h^t| + 1}$ is the inverse document frequency. Let λ_h be the tf-idf of h and $\tilde{\mathcal{D}}_{\mathbb{W}}^t$ be a set consisting of resampled samples of $\mathbb{W}$ in Θ_t . For statistical robustness, we calculate λ_h as the median of $\text{tf-idf}(h, \mathcal{X}_j, \mathcal{G}_h^t)$ across $\tilde{\mathcal{D}}_{\mathbb{W}}^t$. We choose top $N_{\mathcal{A}}$ host $h \in \mathcal{H}_{\mathbb{W}}$ with the highest λ_h to construct $\mathbb{W}$'s anchor host set $\mathcal{A} = \{h_k\}_{k=1}^{N_{\mathcal{A}}}$. Network flows linked to hosts in $\mathcal{A}$ are referred to as anchor flows of $\mathbb{W}$.

• **Bilevel Recognition Models.** We represent the inter-flow pattern of $\mathcal{X}_j$ using a n_z -dimensional feature vector $\mathbf{z}_j$, where $n_z = |\mathcal{A}| + 1$. To derive $\mathbf{z}_j$, we train a binary RF classifier $f_{\mathbb{W}}^t(\mathbf{x})$ on $\tilde{\mathcal{D}}_{\mathbb{W}}^t$ (positive samples) and $\mathcal{D}_{\text{non-}\mathbb{W}}^t = \cup_{w \in \mathcal{W}_t} \mathcal{D}_w^t$ (negative samples) to identify how likely a network flow $\mathbf{x}$ is from $\mathbb{W}$. Besides, we train a multi-class RF classifier $f_{\mathcal{A}}^t(\mathbf{x})$ on $\tilde{\mathcal{D}}_{\mathbb{W}}^t$ to determine whether $\mathbf{x}$ is an anchor flow of $\mathbb{W}$ and, if so, to identify the linked remote host. Specifically, $\mathbf{x}$ is identified to be linked to h_k if $f_{\mathcal{A}}^t(\mathbf{x}) = k$. By leveraging low-level models $f_{\mathbb{W}}^t(\mathbf{x})$, $f_{\mathcal{A}}^t(\mathbf{x})$, and $Q^t(\mathbf{x})$, we derive $\mathbf{z}_j$ by

$$\mathbf{z}_j(k) = \begin{cases} \frac{\sum_{\mathbf{x} \in \mathcal{X}_j} f_{\mathbb{W}}^t(\mathbf{x}) \cdot \mathbf{1}_{f_{\mathcal{A}}^t(\mathbf{x})=k}(\mathbf{x})}{\sum_{\mathbf{x} \in \mathcal{X}_j} \mathbf{1}_{f_{\mathcal{A}}^t(\mathbf{x})=k}(\mathbf{x})}, & k < n_z, \\ \frac{\sum_{\mathbf{x} \in \mathcal{X}_j} f_{\mathbb{W}}^t(\mathbf{x}) \cdot \mathbf{1}_{Q^t(\mathbf{x}) \neq 1}(\mathbf{x})}{\sum_{\mathbf{x} \in \mathcal{X}_j} \mathbf{1}_{Q^t(\mathbf{x}) \neq 1}(\mathbf{x})}, & k = n_z. \end{cases} \quad (3)$$

At the high level, we train a binary RF classifier $\Psi_{\mathbb{W}}^t(\mathbf{z}_j)$ to take $\mathbf{z}_j$ as the input and recognize whether $\mathcal{X}_j$ is from $\mathbb{W}$. We derive the training set $\mathcal{Z} = \{(\mathbf{z}_j, y_j)\}_{j=1}^{M_z}$ for $\Psi_{\mathbb{W}}^t$ using low-level models via Leave-One-Out Cross-Validation [35].

• **Inter-Flow Data Augmentation.** In practice, network flows linked to an anchor host can exhibit diverse patterns. Consequently, the traffic patterns of anchor flows observed in recognition stage might be different from those seen during training, potentially leading to their misidentification. This is particularly relevant in few-shot learning scenarios. To

mitigate the misidentification of anchor flows, we carry out data augmentation by intentionally involving scenarios where some anchor flows fail to be identified. To this end, we first oversample M'_z samples from $\mathcal{Z}$ to construct the expanded sample set $\mathcal{Z}'$. For $(\mathbf{z}_j, y_j) \in \mathcal{Z}'$, we manipulate $\mathbf{z}_j$ as follows

$$\mathbf{z}_j(k) = \begin{cases} \frac{\sum_{\mathbf{x} \in \mathcal{X}_j} f_{\mathbb{W}}^t(\mathbf{x}) \mathbf{m}_j(k) \cdot \mathbf{1}_{f_{\mathcal{A}}^t(\mathbf{x})=k}(\mathbf{x})}{\sum_{\mathbf{x} \in \mathcal{X}_j} \mathbf{m}_j(k) \cdot \mathbf{1}_{f_{\mathcal{A}}^t(\mathbf{x})=k}(\mathbf{x})}, & k < n_z, \\ \frac{\sum_{\mathbf{x} \in \mathcal{X}_j} f_{\mathbb{W}}^t(\mathbf{x}) \mathbf{m}_j(f_{\mathcal{A}}^t(\mathbf{x})) \cdot \mathbf{1}_{Q^t(\mathbf{x}) \neq 1}(\mathbf{x})}{\sum_{\mathbf{x} \in \mathcal{X}_j} \mathbf{1}_{Q^t(\mathbf{x}) \neq 1}(\mathbf{x})}, & k = n_z, \end{cases}$$

where $\mathbf{m}_j$ is a randomly generated mask vector and $\mathbf{m}_j(k) \sim \text{Bernoulli}(1 - \rho_{da})$ simulates the misidentification of network flows linked to h_k with a probability of ρ_{da} .

We train $\Psi_{\mathbb{W}}^t$ on the expanded training set $\mathcal{Z}'$. In recognition stage, we derive the feature vector $\mathbf{z}_j$ of $\mathcal{X}_j \in \mathcal{T}$ using low-level models $f_{\mathbb{W}}^t$, $f_{\mathcal{A}}^t$, and Q^t . Given a recognition threshold ϵ_f , we recognize that $\mathcal{X}_j$ is from $\mathbb{W}$ if $\Psi_{\mathbb{W}}^t(\mathbf{z}_j) \geq \epsilon_f$.

VI. EVALUATION

We evaluate X-EPRINT through extensive experiments. Our experiments aim at answering three research questions.

- **RQ1:** Can X-EPRINT, when trained on traffic samples from one environment, effectively recognize monitored websites from encrypted traffic generated in a different environment?
- **RQ2:** How will X-EPRINT perform in recognizing monitored websites when trained on a very small number of traffic samples from the same environment?
- **RQ3:** To what extent does potential-aware traffic resampling improve the recognition accuracy of X-EPRINT under varying resource constraints?

A. Dataset Construction

To answer the above research questions, we collect encrypted website traffic from diverse environments in our testbed, compensating for the absence of publicly available datasets specific to cross-environmental website fingerprinting.

- **CE-450×6 dataset:** We construct this dataset with a focus on environmental factors, including browsers and proxy software. As shown in Table I, we set up six distinct environments corresponding to different browsers and proxy software to collect encrypted traffic of 450 popular websites according to the rank in Alexa [36]. We automate visits to websites using Selenium [33], setting a maximum wait time of 30 seconds for complete page loading. We capture the encrypted traffic between the encrypted proxy client and server with Tcpdump [34] and record the remote host linked to each network flow from proxy software logs. We collect 50 traffic samples per website in each environment.

TABLE I: Different environments involved in the evaluation.

Environment ID	Proxy Software	Browser
SC	Shadowsocks	Google Chrome
SE	Shadowsocks	Microsoft Edge
SF	Shadowsocks	Mozilla Firefox
VC	V2Ray	Google Chrome
VE	V2Ray	Microsoft Edge
VF	V2Ray	Mozilla Firefox

B. Baseline

To the best of our knowledge, X-EPRINT is the first cross-environmental WF method. We compare it with WF methods:

- **CAWF:** It is one of the SOTA WF methods for encrypted proxies [10], directly relevant to the context of our work.
- **TF:** It constitutes one of the SOTA WF methods for few-shot learning. By leveraging metric learning, TF learns a low-dimensional feature representation and recognizes websites using a KNN classifier [12].
- **Burst+RF (BRF):** It utilizes 16-dimensional raw burst sizes as features and employs an RF classifier for website recognition. BRF is an ablated version of X-EPRINT without inter-flow data augmentation for few-shot environment adaptation.
- **Extended Features+RF (ERF):** It trains an RF classifier with 167-dimensional features (§ III-B) for website recognition. ERF is an ablated version of X-EPRINT without invariant features in zero-shot cross-environmental recognition.

C. Zero-Shot Cross-environmental Recognition

In this experiment, we evaluate how X-EPRINT performs when it is trained on traffic samples from one environment but recognizes monitored websites in another environment.

- **Experimental Setup.** Given a source environment Θ_s and a target environment Θ_t , we randomly involve 50 websites from CE-450×6 dataset to the set $\mathcal{W}_s \cap \mathcal{W}_t$ for generating cross-environmentally invariant features. We consider the remaining 400 websites as monitored websites for which training samples in Θ_t are not available. Our experiments are conducted in the *open-world* setting [7]–[18], which presents a more realistic but challenging scenario compared to the closed-world setting. Specifically, for a monitored website, say $\mathbb{W}$, we construct its training set by involving all 50 traffic samples of $\mathbb{W}$ from Θ_s as positive samples and those of websites in $\mathcal{W}_s \cap \mathcal{W}_t$ from Θ_s as negative samples. In the recognition stage, we construct its testing set by incorporating traffic samples of $\mathbb{W}$ in Θ_t as positive samples and those of another 50 randomly chosen websites which are absent during model training as negative samples. As such, websites considered for negative samples for testing are *completely* different from those for model training.

Both positive and negative testing samples are from Θ_t . For each website, including $\mathbb{W}$ and 50 others, we randomly select 40 instead of all 50 traffic samples to construct the testing set, resulting in 2040 samples for $\mathbb{W}$. This aligns with the subsequent experiment (§ VI-D), where a maximum of 10 traffic samples from Θ_t are used for model training, and the remaining 40 are used for testing. To simulate random website visits by humans, we shuffle and concatenate traffic samples in the testing set in random order to create the encrypted traffic $\mathcal{T}$. For each cross-environmental scenario, we fine-tune an optimal recognition threshold for X-EPRINT, BRF, and ERF respectively to maximize the average F1-score. As for CAWF and TF, we employ their default hyper-parameters [10], [12].

- **Result.** Tables II and III report experimental results for different environmental factors. X-EPRINT consistently outperforms all baseline methods across various scenarios. This is

TABLE II: Evaluating X-EPRINT in cross-browser recognition (mean±standard deviation of F1-score).

Method	Shadowsocks						V2Ray						Average
	SC→SF	SC→SE	SF→SC	SF→SE	SE→SC	SE→SF	VC→VF	VC→VE	VF→VC	VF→VE	VE→VC	VE→VF	
CAWF	0.174 (±0.143)	0.581 (±0.283)	0.342 (±0.255)	0.338 (±0.254)	0.491 (±0.280)	0.157 (±0.131)	0.138 (±0.137)	0.588 (±0.286)	0.213 (±0.215)	0.185 (±0.218)	0.607 (±0.295)	0.117 (±0.147)	0.328 (±0.183)
TF	0.108 (±0.119)	0.484 (±0.269)	0.112 (±0.143)	0.120 (±0.140)	0.492 (±0.275)	0.139 (±0.157)	0.137 (±0.141)	0.504 (±0.245)	0.132 (±0.140)	0.149 (±0.172)	0.500 (±0.264)	0.149 (±0.146)	0.252 (±0.172)
BRF	0.260 (±0.337)	0.646 (±0.300)	0.337 (±0.337)	0.373 (±0.316)	0.682 (±0.333)	0.342 (±0.349)	0.364 (±0.350)	0.740 (±0.287)	0.482 (±0.339)	0.430 (±0.338)	0.784 (±0.280)	0.338 (±0.354)	0.482 (±0.174)
ERF	0.237 (±0.330)	0.626 (±0.303)	0.308 (±0.329)	0.363 (±0.320)	0.723 (±0.332)	0.351 (±0.370)	0.360 (±0.360)	0.764 (±0.274)	0.502 (±0.344)	0.458 (±0.329)	0.807 (±0.274)	0.362 (±0.357)	0.488 (±0.186)
X-EPRINT	0.794 (±0.301)	0.901 (±0.180)	0.713 (±0.308)	0.735 (±0.290)	0.870 (±0.236)	0.695 (±0.334)	0.572 (±0.338)	0.860 (±0.206)	0.626 (±0.316)	0.574 (±0.325)	0.833 (±0.209)	0.571 (±0.330)	0.729 (±0.118)

TABLE III: Evaluating X-EPRINT in cross-proxy recognition (mean±standard deviation of F1-score).

Method	Google Chrome		Microsoft Edge		Mozilla Firefox		Average
	SC→VC	VC→SC	SE→VE	VE→SE	SF→VF	VF→SF	
CAWF	0.377 (±0.312)	0.423 (±0.313)	0.387 (±0.273)	0.480 (±0.332)	0.087 (±0.139)	0.074 (±0.094)	0.305 (±0.162)
TF	0.403 (±0.274)	0.388 (±0.295)	0.359 (±0.248)	0.395 (±0.266)	0.207 (±0.223)	0.261 (±0.253)	0.336 (±0.072)
BRF	0.188 (±0.233)	0.475 (±0.298)	0.232 (±0.255)	0.451 (±0.324)	0.147 (±0.208)	0.313 (±0.316)	0.301 (±0.125)
ERF	0.258 (±0.285)	0.571 (±0.316)	0.330 (±0.277)	0.536 (±0.314)	0.204 (±0.269)	0.419 (±0.332)	0.386 (±0.136)
X-EPRINT	0.685 (±0.280)	0.795 (±0.256)	0.679 (±0.257)	0.779 (±0.252)	0.529 (±0.320)	0.739 (±0.288)	0.701 (±0.088)

TABLE IV: Evaluating X-EPRINT in few-shot environment adaptation (mean±standard deviation of F1-score).

N-Shot	Method	SC	SE	SF	VC	VE	VF	Average
3-Shot	CAWF	0.612 (±0.192)	0.511 (±0.236)	0.359 (±0.170)	0.546 (±0.196)	0.571 (±0.174)	0.312 (±0.165)	0.485 (±0.111)
	TF	0.535 (±0.257)	0.422 (±0.256)	0.312 (±0.196)	0.546 (±0.225)	0.404 (±0.194)	0.335 (±0.167)	0.426 (±0.090)
	BRF	0.924 (±0.137)	0.911 (±0.140)	0.869 (±0.198)	0.802 (±0.216)	0.789 (±0.221)	0.752 (±0.308)	0.841 (±0.064)
	ERF	0.912 (±0.147)	0.898 (±0.156)	0.839 (±0.203)	0.815 (±0.204)	0.810 (±0.204)	0.752 (±0.285)	0.838 (±0.054)
	X-EPRINT	0.958 (±0.112)	0.939 (±0.129)	0.944 (±0.126)	0.905 (±0.152)	0.879 (±0.181)	0.927 (±0.145)	0.925 (±0.026)
5-Shot	CAWF	0.671 (±0.177)	0.607 (±0.218)	0.404 (±0.190)	0.649 (±0.176)	0.637 (±0.196)	0.351 (±0.179)	0.553 (±0.127)
	TF	0.592 (±0.211)	0.503 (±0.212)	0.354 (±0.264)	0.601 (±0.171)	0.492 (±0.198)	0.408 (±0.231)	0.492 (±0.090)
	BRF	0.951 (±0.102)	0.941 (±0.111)	0.898 (±0.164)	0.854 (±0.174)	0.850 (±0.192)	0.800 (±0.281)	0.882 (±0.053)
	ERF	0.946 (±0.110)	0.932 (±0.115)	0.896 (±0.171)	0.865 (±0.170)	0.857 (±0.188)	0.806 (±0.278)	0.884 (±0.043)
	X-EPRINT	0.965 (±0.105)	0.941 (±0.123)	0.951 (±0.121)	0.928 (±0.132)	0.902 (±0.145)	0.947 (±0.106)	0.939 (±0.020)
10-Shot	CAWF	0.780 (±0.154)	0.704 (±0.207)	0.489 (±0.176)	0.767 (±0.129)	0.715 (±0.161)	0.396 (±0.193)	0.642 (±0.146)
	TF	0.610 (±0.218)	0.529 (±0.239)	0.357 (±0.206)	0.654 (±0.172)	0.522 (±0.236)	0.433 (±0.273)	0.518 (±0.100)
	BRF	0.968 (±0.078)	0.970 (±0.066)	0.938 (±0.127)	0.904 (±0.141)	0.900 (±0.154)	0.830 (±0.258)	0.918 (±0.048)
	ERF	0.966 (±0.081)	0.958 (±0.082)	0.925 (±0.137)	0.909 (±0.137)	0.903 (±0.143)	0.837 (±0.255)	0.916 (±0.042)
	X-EPRINT	0.977 (±0.063)	0.970 (±0.067)	0.971 (±0.081)	0.953 (±0.095)	0.935 (±0.120)	0.965 (±0.081)	0.962 (±0.014)

because baseline methods operate under the implicit assumption that both training and recognition samples are from the same environment, making them sensitive to feature changes across environments. Compared to BRF and ERF, CAWF and TF perform even worse because their complex models are prone to overfitting the training samples from Θ_s . Contrarily, X-EPRINT is substantially more robust against the feature drift across environments. It achieves reasonable accuracy in cross-browser recognition with an F1-score of 0.729, which is 49.4% higher than that of ERF, the baseline with the best performance. The advantage of X-EPRINT is more significant in cross-proxy recognition, where its F1-score exceeds that of ERF by 81.6%. Synthesizing results from Tables II and III, X-EPRINT achieves an average F1-score of 0.719, which is 58.4% higher than the top-performing baseline method ERF. Despite its overall advantage, X-EPRINT shows somewhat diminished performance with Firefox as the selected browser. A closer look at Fig. 1 reveals the underlying cause: feature drift with Firefox is typically more significant than with other browsers in both cross-browser and cross-proxy scenarios.

• **Environment Identification.** The performance of X-EPRINT in Tables II and III, is based on the precondition that it had

successfully identified Θ_t . We further evaluate whether X-EPRINT is able to accurately identify Θ_t . We randomly select 50 websites to train X-EPRINT for environment identification, using the remaining 400 for testing. Therefore, the training and testing websites are disjoint, preventing the model from overfitting to website-specific features and ensuring generalization to unseen websites. To generate the testing traffic in Θ_t , we include varying numbers of traffic samples from testing websites within Θ_t to synthesize $\mathcal{T}$. For statistical soundness, we repeat test traffic generation and environment identification 100 times and report the average performance in Table V. We observe a dramatic increase in the F1-score as the number of samples increases, reaching a perfect F1-score of 1.000 with 200 samples for all target environments. In previous experiments (Tables II and III), $\mathcal{T}$ consists of 2040 traffic samples, far exceeding 200, which suggests that X-EPRINT accurately identifies target environments.

Answer to RQ1: X-EPRINT substantially outperforms baseline methods in zero-shot cross-environmental recognition. It achieves an average F1-score of 0.719, which is 58.4% higher than the top-performing baseline method.

TABLE V: Environment identification results (F1-score).

Sample Count	SC	SE	SF	VC	VE	VF
1	0.966	0.958	0.979	0.948	0.941	0.962
10	0.976	0.973	0.979	0.976	0.972	0.970
60	0.991	0.990	0.989	0.991	0.989	0.986
100	0.996	0.994	0.993	0.993	0.992	0.991
200	1.000	1.000	1.000	1.000	1.000	1.000

D. Few-Shot Environment Adaptation

We next evaluate whether X-EPRINT is able to accurately recognize monitored websites when trained on a very small number of traffic samples from the target environment.

- **Experimental Setup.** We also consider the open-world setting. Similar to the experimental setup in § VI-C, we randomly include 50 websites from CE-450×6 dataset to the set $\mathcal{W}_t$, whereas the remaining 400 websites are considered as monitored websites. Assume that the monitored website $\mathbb{W}$ has been chosen for resampling N traffic samples within Θ_t . To recognize $\mathbb{W}$, we construct its training set by involving these N new traffic samples as positive samples and those of websites in $\mathcal{W}_t$ from Θ_t as negative samples. The construction of testing set is consistent with that in § VI-C. Recognition threshold is fine-tuned for specific target environment.

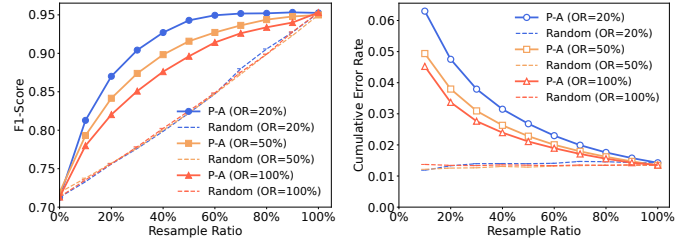
- **Result.** Table IV presents the experimental results. Compared to zero-shot cross-environmental recognition, all methods significantly enhance recognition accuracy, even with a limited number of training samples from the target environment. Generally, the performance of all methods improves as the number of training samples N increases. X-EPRINT consistently achieves higher F1-score than baseline methods in all target environments. Its advantage is more significant when N is smaller due to its adoption of inter-flow data augmentation. For example, with just 3 new traffic samples resampled in the target environment, X-EPRINT achieves an average F1-score of 0.925, improving by approximately 0.084 over the top-performing baseline method BRF.

Answer to RQ2: X-EPRINT achieves few-shot environment adaptation with reasonable accuracy in website recognition, attaining an average F1-score of 0.925 in 3-shot scenarios.

E. Potential-Aware Traffic Resampling

We finally experimentally analyze to what extent potential-aware traffic resampling improves X-EPRINT's performance.

- **Experimental Setup.** We still follow the open-world setting. Let M be the number of monitored websites and η_r be the resample ratio. Given Θ_s and Θ_t , X-EPRINT carries out cross-environmental WF in a three-stage pipeline. It first recognizes monitored websites using models trained without samples from Θ_t . Based on the confidence scores, X-EPRINT selectively chooses $M\eta_r$ monitored websites to resample N traffic samples for each in Θ_t . If a monitored website is resampled, X-EPRINT utilizes the new samples to perform few-shot environment adaptation and re-recognizes it to update the recognition results. In practice, not all M monitored websites must appear in $\mathcal{T}$, and the frequency with which a monitored



(a) Improving the F1-score of website recognition via traffic resampling. (b) Prioritizing the resampling of websites with high error rate.

Fig. 3: Evaluating potential-aware traffic resampling.

website is visited varies. Let r_o be the occurrence rate (OR) of a monitored website in $\mathcal{T}$. We denote by n_i the frequency of visits to the monitored website $\mathbb{W}_i$ within $\mathcal{T}$. If $\mathbb{W}_i$ is absent in $\mathcal{T}$, we have $n_i = 0$. Otherwise, we assume that $n_i \sim \text{Uniform}(1, 40)$ following uniform distribution.

Given that the recognition of each monitored website is treated as a binary classification task, the ratio of positive to negative samples in $\mathcal{T}$ can be computed by $r = \frac{\sum_{i=1}^M n_i}{\sum_{i=1}^M |\mathcal{T}| - n_i}$. We adjust the number of samples in $\mathcal{T}$, i.e., $|\mathcal{T}|$, to meet $r = 1 : 50$, aligning with the settings of experiments in § VI-C and § VI-D. In addition to traffic samples from monitored websites, we include $|\mathcal{T}| - \sum_{i=1}^M n_i$ traffic samples from unmonitored websites in $\mathcal{T}$. We set $M = 20$ and $N = 10$. Therefore, we randomly select 20 websites from 400 websites that are not in $\mathcal{W}_s \cap \mathcal{W}_t$ as monitored websites and the other 380 websites as unmonitored websites to generate $\mathcal{T}$ and evaluate X-EPRINT. We repeat this process 20 times to report the average performance, ensuring that the monitored websites do not overlap across 20 iterations to cover all 400 websites. Besides, random traffic resampling (i.e., randomly choosing monitored websites for resampling) is involved as a reference. We use the micro F1-score, rather than the macro F1-score used in previous experiments, to prevent ill-posed results due to the absence of some monitored websites in $\mathcal{T}$.

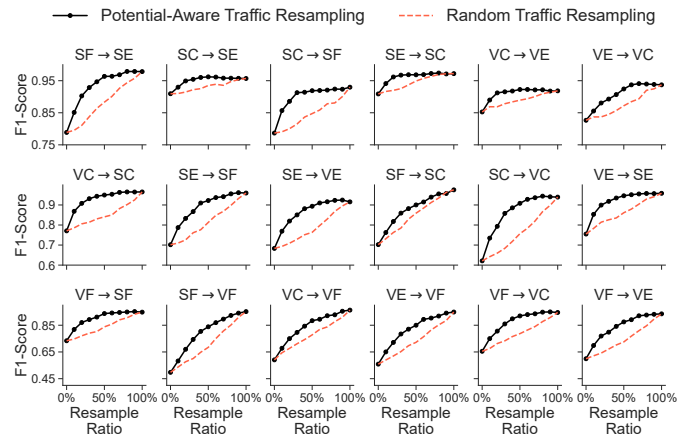


Fig. 4: F1-score improvement through resampling in different cross-environmental scenarios (OR = 50%).

- **Result.** Fig. 3 reports results aggregated across 18 cross-environmental scenarios. In Fig. 3(a), the F1-score increases approximately linearly with the resample ratio η_r for random

traffic resampling. However, the potential-aware (P-A) traffic resampling utilized by X-EPRINT dramatically accelerates the rise of F1-score, significantly outperforming random traffic resampling. For example, given 50% OR of monitored websites, P-A traffic resampling improves F1-score by 0.0741 via resampling only 10% monitored websites, which is 3.97 times the improvement made by random traffic resampling. Fig. 4 illustrates how F1-score changes with the resample ratio in 18 scenarios, given 50% OR of monitored websites. P-A traffic resampling consistently surpasses random traffic resampling, demonstrating a substantially faster upward trend in F1-score improvement. The effectiveness of P-A traffic resampling stems from its ability to identify websites with high error rates in zero-shot cross-environmental recognition and prioritize them for traffic resampling as shown in Fig 3(b). It suggests that P-A traffic resampling will be more effective if error rates of monitored websites vary significantly. Error rate variation can be characterized by the coefficient of variation (CV). A higher CV indicates greater variability in error rates, and P-A traffic resampling tends to be more effective under these conditions. The average CVs for OR = 20%, 50%, 100% are 0.516, 0.441, and 0.397 respectively, coinciding with the results illustrated in Fig. 3, where P-A traffic resampling yields the best performance at OR = 20%, with OR = 50% and OR = 100% following in effectiveness.

Answer to RQ3: Potential-aware traffic resampling significantly enhances X-EPRINT's performance, boosting the F1-score by 0.0741 by resampling 10% of monitored websites, a 3.97-fold increase over random traffic resampling.

VII. RELATED WORK

Traffic fingerprinting (TF), a key traffic analysis technique, infers user behavior without accessing packet payload plaintext, making it resilient to encryption. Considering various types of traffic, TF methods can be classified into three major categories: website fingerprinting [7]–[20], [37]–[39], app fingerprinting [40]–[45], and IoT fingerprinting [46]–[52].

- **Website Fingerprinting.** Our work falls into this category. Website fingerprinting (WF) aims to infer the websites visited via encrypted proxies or anonymity networks [3]–[5]. Existing WF methods can be roughly grouped into feature-engineering-based WF and deep-learning-based WF. The first category extracts features based on domain knowledge and trains classifiers to recognize websites [14]–[18]. For example, Panchenko et al. sampled features from the cumulative representation of traces and trained an SVM for website fingerprinting [15]; Ma et al. extracted spatial-temporal features to recognize websites [10]. The second category makes use of deep learning models to achieve automatic feature extraction [9], [13], [19]. For examples, Sirinam et al. achieved high-precision website fingerprinting using CNN [9]. Despite their success, existing WF methods cannot tackle traffic feature drift across different environments. Feature engineering-based WF methods heavily rely on domain-specific knowledge for feature extraction. It is extremely labor-intensive to manually extract invariant features

for numerous cross-environmental scenarios. Deep learning-based WF methods, with a large number of parameters, are particularly susceptible to overfitting, increasing their sensitivity to cross-environmental feature drift.

- **Concept Drift Mitigation.** Feature drift related to cross-environmental WF is a type of concept drift, which is still an open problem for (deep) machine learning [24]–[26]. To alleviate concept drift, there are two mainstream approaches. The first one involves periodic sample collection and model retraining [19], [53]–[55]. However, it focuses on concept drift from temporal evolution and fails to effectively address cross-environmental feature drift in our problem due to entanglement in the sampling dilemma. The second approach is to detect drift samples and perform manual relabeling [24], [25], [56], [57]. Unfortunately, it is unsuitable for WF as encrypted traffic often cannot be decrypted to obtain ground truth.

- **Few-Shot Adaptation.** WF methods, particularly those that rely on deep learning, e.g., [9], [13], are generally data-intensive, which limits their ability to quickly adapt to new tasks or environments. Some advanced WF methods, utilizing metric learning [12], [20] and adversarial domain adaptation [39], support few-shot website fingerprinting. However, these methods are tailored for Tor traffic [3] and are not effectively applicable to our problem, which focuses on encrypted proxies [4], [5]. This is because traffic samples in our scenario typically comprise a varied number of network flows, hindering proper sample alignment for these methods.

VIII. CONCLUSION

We took the first step to cross-environmental website fingerprinting and advanced a systematic framework dubbed X-EPRINT. Extensive experimental results demonstrate the substantial advantage of X-EPRINT over baseline methods in the cross-environmental website recognition. Even in the absence of training samples from monitored websites in the target environment, X-EPRINT effectively recognizes these websites from encrypted traffic, achieving an average F1-score of 0.719, which is 58.4% higher than that of the top-performing baseline method. Its average F1-score is further improved to 0.925 via resampling just 3 training samples in the target environment. The authors have provided public access to their code and data at <https://github.com/cry4tall/xepriint>.

ACKNOWLEDGMENT

This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFB3107800, National Natural Science Foundation (62202405, 62411560154, 62272381, U23A20332, and T2341003), Natural Science Basic Research Program of Shaanxi Province (2023-JC-JQ-50), the Fundamental Research Funds for the Central Universities, Postdoctoral Science Foundation (2024T170722, 2023M732791), Hong Kong RGC (Research Grants Council) Project (No. PolyU15224121), and HKPolyU Project (No. 1-ZVG0), of China.

REFERENCES

- [1] H. Lee, D. Kim, and Y. Kwon, "Tls 1.3 in practice: How tls 1.3 contributes to the internet," in *ACM WWW*, 2021.
- [2] E. Rescorla, "The transport layer security (tls) protocol version 1.3," Tech. Rep., 2018.
- [3] C. Wang, J. Luo, Z. Ling, L. Luo, and X. Fu, "A comprehensive and long-term evaluation of tor v3 onion services," in *IEEE INFOCOM*, 2023.
- [4] "Shadowsocks," <https://zh.wikipedia.org/wiki/Shadowsocks>, 2024.
- [5] "V2Ray," <https://zh.wikipedia.org/wiki/V2Ray>, 2024.
- [6] Q. Ji, Z. Rao, M. Chen, and J. Luo, "Security analysis of shadowsocks (r) protocol," *SCN*, 2022.
- [7] M. Shen, K. Ji, Z. Gao, Q. Li, L. Zhu, and K. Xu, "Subverting website fingerprinting defenses with robust traffic representation," in *USENIX Security*, 2023.
- [8] Y. Wang, H. Xu, Z. Guo, Z. Qin, and K. Ren, "Snwf: website fingerprinting attack by ensembling the snapshot of deep learning," *IEEE TIFS*, 2022.
- [9] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *ACM CCS*, 2018.
- [10] X. Ma, M. Shi, B. An, J. Li, D. X. Luo, J. Zhang, and X. Guan, "Context-aware website fingerprinting over encrypted proxies," in *IEEE INFOCOM*, 2021.
- [11] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *USENIX Security*, 2014.
- [12] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning," in *ACM CCS*, 2019.
- [13] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," *arXiv*, 2017.
- [14] T. Wang, "High precision open-world website fingerprinting," in *IEEE S&P*, 2020.
- [15] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *NDSS*, 2016.
- [16] T. Wang and I. Goldberg, "On realistically attacking tor with website fingerprinting," *PoPETs*, 2016.
- [17] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *USENIX Security*, 2016.
- [18] A. Shusterman, L. Kang, Y. Haskal, Y. Meltser, P. Mittal, Y. Oren, and Y. Yarom, "Robust website fingerprinting through the cache occupancy channel," in *USENIX Security*, 2019.
- [19] X. Deng, Q. Yin, Z. Liu, X. Zhao, Q. Li, M. Xu, K. Xu, and J. Wu, "Robust multi-tab website fingerprinting attacks in the wild," in *IEEE S&P*, 2023.
- [20] Y. Xie, J. Feng, W. Huang, Y. Zhang, X. Sun, X. Chen, and X. Luo, "Contrastive fingerprinting: A novel website fingerprinting attack over few-shot traces," in *ACM WWW*, 2024.
- [21] U. Iqbal, S. Englehardt, and Z. Shafiq, "Fingerprinting the fingerprinters: Learning to detect browser fingerprinting behaviors," in *IEEE S&P*, 2021.
- [22] X. Li, B. A. Azad, A. Rahmati, and N. Nikiforakis, "Good bot, bad bot: Characterizing automated browsing activity," in *IEEE S&P*, 2021.
- [23] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Computing Surveys*, 2021.
- [24] L. Yang, W. Guo, Q. Hao, A. Ciptadi, A. Ahmadzadeh, X. Xing, and G. Wang, "{CADE}: Detecting and explaining concept drift samples for security applications," in *USENIX Security*, 2021.
- [25] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "Insomnia: Towards concept-drift robustness in network intrusion detection," in *ACM AISec*, 2021.
- [26] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nouredinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in *USENIX Security*, 2017.
- [27] "Internet browsers: how many are there?" <https://www.meetsidekick.com/internet-browsers-how-many-are-there/>, 2024.
- [28] P. R. Silva, J. Vinagre, and J. Gama, "A dtw approach for complex data a case study with network data streams," in *ACM/SIGAPP SAC*, 2023.
- [29] M. T. Alam, R. Fieblinger, A. Mahara, and N. Rastogi, "Morph: Towards automated concept drift adaptation for malware detection," *arXiv*, 2024.
- [30] F. Kong, Y. Yang, W. Chen, and S. Li, "The hardness analysis of thompson sampling for combinatorial semi-bandits with greedy oracle," *NIPS*, 2021.
- [31] P. Perrault, E. Boursier, M. Valko, and V. Perchet, "Statistical efficiency of thompson sampling for combinatorial semi-bandits," *NIPS*, 2020.
- [32] P. Perrault, "When combinatorial thompson sampling meets approximation regret," *NIPS*, 2022.
- [33] "Selenium," <https://github.com/SeleniumHQ/selenium>, 2024.
- [34] "Tcpdump," <https://www.tcpdump.org>, 2024.
- [35] T.-T. Wong, "Performance evaluation of classification algorithms by k-fold and leave-one-out cross validation," *Pattern recognition*, 2015.
- [36] "Alexa Internet," https://en.wikipedia.org/wiki/Alexa_Internet, 2024.
- [37] Q. Yin, Z. Liu, Q. Li, T. Wang, Q. Wang, C. Shen, and Y. Xu, "An automated multi-tab website fingerprinting attack," *IEEE TDSC*, 2021.
- [38] G. Cherubin, R. Jansen, and C. Troncoso, "Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world," in *USENIX Security*, 2022.
- [39] C. Wang, J. Dani, X. Li, X. Jia, and B. Wang, "Adaptive fingerprinting: Website fingerprinting over few encrypted traffic," in *ACM CODASPY*, 2021.
- [40] J. Li, H. Zhou, S. Wu, X. Luo, T. Wang, X. Zhan, and X. Ma, "Foap: fine-grained open-world android app fingerprinting," in *USENIX Security*, 2022.
- [41] J. Li, S. Wu, H. Zhou, X. Luo, T. Wang, Y. Liu, and X. Ma, "Packet-level open-world app fingerprinting on wireless traffic," in *NDSS*, 2022.
- [42] T. van Ede, R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Hoffnes, M. van Steen, and A. Peter, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *NDSS*, 2020.
- [43] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE TIFS*, 2017.
- [44] T. Ni, G. Lan, J. Wang, Q. Zhao, and W. Xu, "Eavesdropping mobile app activity via {Radio-Frequency} energy harvesting," in *USENIX Security*, 2023.
- [45] S. Oh, M. Lee, H. Lee, E. Bertino, and H. Kim, "Appsniiffer: Towards robust mobile app fingerprinting against vpn," in *ACM WWW*, 2023.
- [46] X. Ma, J. Qu, J. Li, J. C. Lui, Z. Li, and X. Guan, "Pinpointing hidden iot devices via spatial-temporal traffic fingerprinting," in *IEEE INFOCOM*, 2020.
- [47] Y. Wan, K. Xu, G. Xue, and F. Wang, "Totargos: A multi-layer security monitoring system for internet-of-things in smart homes," in *IEEE INFOCOM*, 2020.
- [48] X. Ma, J. Qu, J. Li, J. C. Lui, Z. Li, W. Liu, and X. Guan, "Inferring hidden iot devices and user interactions via spatial-temporal traffic fingerprinting," *IEEE/ACM TON*, 2021.
- [49] C. Kuzniar, M. Neves, V. Gurevich, and I. Haque, "Poiriot: Fingerprinting iot devices at tbps scale," *IEEE/ACM TON*, 2024.
- [50] M. K. Fadul, D. R. Reising, L. P. Weerasena, T. D. Loveless, M. Sartipi, and J. H. Tyler, "Improving rf-dna fingerprinting performance in an indoor multipath environment using semi-supervised learning," *IEEE TIFS*, 2024.
- [51] R. Trimananda, J. Varmarken, A. Markopoulou, and B. Demsky, "Packet-level signatures for smart home devices," in *NDSS*, 2020.
- [52] A. Nascita, F. Cerasuolo, D. Di Monda, J. T. A. Garcia, A. Montieri, and A. Pescapè, "Machine and deep learning approaches for iot attack classification," in *IEEE INFOCOM*, 2022.
- [53] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "{TESSERACT}: Eliminating experimental bias in malware classification across space and time," in *USENIX Security*, 2019.
- [54] D. Nigenda, Z. Karnin, M. B. Zafar, R. Ramesha, A. Tan, M. Donini, and K. Kenthapadi, "Amazon sagemaker model monitor: A system for real-time insights into deployed machine learning models," in *ACM SIGKDD*, 2022.
- [55] S. Han, Q. Wu, H. Zhang, B. Qin, J. Hu, X. Shi, L. Liu, and X. Yin, "Log-based anomaly detection with robust feature extraction and online learning," *IEEE TIFS*, 2021.
- [56] D. Han, Z. Wang, W. Chen, K. Wang, R. Yu, S. Wang, H. Zhang, Z. Wang, M. Jin, J. Yang *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *NDSS*, 2023.
- [57] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending transcend: Revisiting malware classification in the presence of concept drift," in *IEEE S&P*, 2022.